

Ai-VendorHub

Complete Project Feature Documentation

Project	Full-stack AI-powered e-commerce marketplace
Architecture	Node.js and Express microservices with MongoDB, Redis, RabbitMQ, Socket.IO, React frontends, Razorpay, ImageKit, and Google Gemini/LangChain AI features
Primary users	Customers, sellers, admins, and service operators
Generated	2026-07-03

This document consolidates the major implemented and planned features visible in the Ai-VendorHub codebase. It is intended as a project handoff, demo reference, and feature inventory.

USER	SELLER	ADMIN	AI
Browse, search, wishlist, cart, checkout, orders, payments, notifications	Product management, orders, metrics, inventory analytics, low-stock alerts	Homepage content, notifications, cart/order maintenance, broad product access	Search intent, product insights, chat, comparison, recommendations, budget and mood shopping

Contents

1. Executive Overview
2. Architecture and Data Flow
3. Customer Features
4. Seller and Admin Features
5. AI Features
6. Route Inventory
7. Frontend Experience and Technology Stack
8. Status, Setup, and Notes
9. Source Files Used

1. Executive Overview

Ai-VendorHub is a modular marketplace application that combines standard e-commerce workflows with seller operations and AI-assisted shopping. The repository is organized around independent services for authentication, product catalog, cart, orders, payments, notifications, seller dashboard, and AI.

Core outcomes

- Customers can register, authenticate, manage addresses, browse products, use wishlist/recently-viewed flows, manage carts, create orders, pay, and receive notifications.
- Sellers can create and maintain products, inspect their catalog and orders, monitor metrics, view inventory risk, and handle low-stock events.
- Admins can manage dynamic homepage content, create platform notifications, inspect notifications, and run operational scans such as abandoned carts or order expiry.
- AI capabilities improve discovery and selling workflows through natural-language search, generated descriptions, category/tag suggestions, review summaries, comparison, recommendations, budget optimization, mood shopping, and chat.

High-level service map

Service	Default port	Primary responsibility
Auth	3001	JWT authentication, role-specific login, Google OAuth, profile, address book, password reset, email verification, refresh/logout flows.
Product	3000	Catalog CRUD, image upload, filtering, comparison, trending, homepage content, wishlist, recently viewed, related products, variants.
Cart	3002	Cart CRUD, stock-aware validation, totals, save-for-later, cart status/health, abandoned cart events.
Order	3003	Order creation from cart, order history/details, cancellation, address updates, status updates, expiry scans.
Payment	Not fixed in docs	Razorpay order/payment creation, verification, payment lookup, local test-success flow.
Notification	Not fixed in docs	User/seller/admin notification inbox, unread counts, email retry, read/delete actions.
Seller Dashboard	Separate service	Seller metrics, order/product listings, live feed, analytics, inventory risk/forecasting, low-stock alerts.
AI	3005	Gemini/LangChain powered shopping and product intelligence endpoints.

2. Architecture and Data Flow

The project follows a service-oriented backend with shared JWT authorization patterns and event-driven integrations. Each service owns its route layer, controller logic, models, and persistence wiring.

Client / Frontend	Calls service APIs using JWT access tokens and service-specific endpoints.
Auth Service	Issues and refreshes tokens, validates users, stores profile/address data, and supports Google OAuth when configured.
Product Service	Provides browsable catalog data and seller/admin catalog management. Product image uploads are capped at five files per request.
Cart Service	Stores product snapshots, revalidates stock, calculates totals, and can publish abandoned cart events.
Order Service	Transforms/validates shipping address input, creates orders from cart contents, clears cart, and supports order lifecycle updates.

RabbitMQ	Used by multiple services for inter-service events such as order creation and dashboard/notification updates.
Payment Service	Creates and verifies Razorpay payments and offers a test success route for local development.
Notification + Seller Dashboard	Surface operational events to users, sellers, and admins through inboxes, feeds, metrics, and alerts.
AI Service	Calls product/auth/cart contexts and Gemini-powered logic to enrich shopping and seller workflows.

Typical customer checkout flow

Step	Action	Service behavior
1	Register or login	Auth creates a user session and returns JWT tokens.
2	Browse/search catalog	Product service handles filtering, product detail, related products, trending, and comparisons.
3	Add/update cart	Cart stores item snapshots, checks stock, calculates subtotal/tax/shipping/total, and supports save-for-later.
4	Create order	Order reads cart contents, validates address and stock, creates PENDING order, publishes event, and clears cart.
5	Pay	Payment service creates Razorpay orders, verifies payment, or uses test success locally.
6	Track/notify	Order, notification, and dashboard services surface status changes and events.

3. Customer Features

Area	Features
Authentication	User registration, login, token refresh, current user profile, profile updates, logout from one device, logout from all devices.
Account recovery	Forgot password, reset password, email verification request, email verification by token.
Google OAuth	Google login/callback with configurable role selection for user, seller, or admin when the provider is configured.
Address book	List, add, and delete current user's shipping addresses.
Product discovery	Catalog pagination/filtering, product detail, related products, trending products, product comparison, recently viewed history.
Wishlist	Get wishlist, add product to wishlist, remove product from wishlist.
Cart	Add item, update quantity, remove item, view cart, clear cart, validate cart, cart status, health endpoint, save-for-later, move saved item back to cart.
Orders	Create order, view own orders, get order by ID, cancel pending order, update pending order address.
Payments	Create payment, create Razorpay order, verify payment, get payment by ID, local test success route for development.
Notifications	List own notifications, unread count, view detail, mark one read, mark all read, delete notification.
AI shopping	Search intent parsing, product insights, chat, similar products, comparison, smart budget shopping, mood shopping, review summaries.

4. Seller and Admin Features

Seller workspace

Capability	Details
Seller authentication	Dedicated seller registration and seller login endpoints, with seller role enforcement on protected routes.
Product management	Create products with images, list own products, update own products, delete products. Admins can also manage products broadly.
Catalog enrichment	Seller/admin variant creation and updates, AI generated descriptions, category/tag suggestions, product health analytics.
Orders	Seller dashboard exposes seller order lists and order status update support exists for admin/seller roles.
Metrics	Seller metrics endpoint, live order feed, read tracking for feed events.
Analytics	Conversion funnel, product health dashboard, inventory movement, top-losing products, inventory risk, inventory forecast.
Inventory alerts	Low-stock alert list, mark read, resolve alert.

Admin controls

Capability	Details
Role-specific login	Admin login endpoint uses the same auth validation with role-specific controller behavior.
Homepage management	Create, update, delete, and list homepage sections/banners/product rows from product or seller dashboard services.
Notifications	Create notifications, list all notifications, retry email delivery for failed notification emails.
Operations	Run abandoned-cart scans, expire pending orders, inspect notification inventory, and update order statuses.
Product oversight	Admins can create, update, delete, and manage homepage-related product content.

5. AI Features

The AI service is built around Google Gemini/LangChain components with middleware-protected shopping intelligence endpoints and public seller-assist endpoints.

Endpoint/function	Access	Purpose
POST /ai/search-intent	Authenticated	Parse natural language queries into filters and product matches.
GET /ai/product/:productId/insights	Authenticated	Generate product detail page insights.
POST /ai/chat	Authenticated	Conversational shopping assistant.
GET /ai/similar/:productId	Authenticated	Recommend similar products.
POST /ai/compare	Authenticated	AI-assisted product comparison.
POST /ai/smart-budget	Authenticated	Optimize product choices against a budget.
POST /ai/mood-shopping	Authenticated	Find products from mood or occasion prompts.
POST /ai/generate-description	Public/internal	Generate professional product copy, bullets, tags, and SEO keywords.
POST /ai/suggest-category-tags	Public/internal	Suggest categories and tags with confidence.
POST /ai/review-summary/:productId	Authenticated	Summarize review sentiment, pros, cons, and recommendation score.
GET/POST /ai/feature-flags	Runtime control	Expose and update AI feature flags.
GET /ai/metrics and /ai/scope	Observability	Expose metrics and allowed AI operating scope.

Reliability and guardrails

- AI utilities include JSON parsing, retry with backoff, circuit breaker, domain guard, feature flags, and metrics helpers.
- Authenticated routes use service middleware before allowing personalized AI shopping actions.
- The service declares dependencies on product, auth, and cart services for richer product-aware responses.

6. Route Inventory

Auth routes

Method	Path	Purpose
GET	/	Auth service health.
POST	/auth/register	Register user.
POST	/auth/register/seller	Register seller.
POST	/auth/login	User login.
POST	/auth/login/admin	Admin login.
POST	/auth/login/seller	Seller login.
POST	/auth/verify-email/request	Request verification email.
GET/POST	/auth/verify-email/token	Verify email token.
POST	/auth/password/forgot	Start password reset.
POST	/auth/password/reset/:token	Reset password.
POST	/auth/refresh	Refresh access token.
GET, PATCH	/auth/me or /auth/users/me	Read or update current user.
GET, POST	/auth/logout	Logout current session.
POST	/auth/logout-all	Logout all devices.
GET	/auth/google and /auth/google/callback	Google OAuth flow.
GET, POST, DELETE	/auth/users/me/addresses	Address list/add/delete.

Product, cart, order, payment, notification, and seller routes

Service	Important paths
Product	/, /seller, /compare, /trending, /homepage, /homepage/admin, /recently-viewed, /wishlist, /:id/view, /:id/related, /:id/variants, /:id
Cart	/health, /items, /items/:productId, /validate, /status, /items/:productId/save-for-later, /save-for-later, /save-for-later/:productId/move-to-cart, /abandoned/scan, /
Order	/, /me, /:id/cancel, /expiry/scan, /:id/status, /:id/address, /:id
Payment	/create/:orderId, /razorpay/order, /verify, /test-success/:orderId, /:id
Notification	/, /unread-count, /admin/all, /, /read-all, /:id/retry-email, /:id, /:id/read, /:id
Seller dashboard	/metrics, /analytics/conversion-funnel, /analytics/product-health, /analytics/inventory-movement, /analytics/top-losing-products, /analytics/inventory-risk, /analytics/inventory-forecast, /orders, /products, /feed, /low-stock-alerts, /homepage

7. Frontend Experience and Technology Stack

Frontend surfaces

Surface	Features
Product frontend	React/Vite marketplace UI with responsive product grid, search, category and price filters, product detail pages, cart/page integrations, checkout page, admin portal, AI assistant components, and seller workspace components.
Auth frontend	React/Vite authentication pages for home, register, login, forgot password, verify email, reset password, profile, and product detail entry points.
Shared UI patterns	Responsive navigation, mobile menus, loading/error/empty states, Tailwind styling, lucide icons, Axios service clients, React Context state management.
AI frontend integrations	AIAssistant, AIChatBot, AIControlCenter, AISmartFilterBanner, product/service clients for AI APIs.

Backend and infrastructure stack

Layer	Technologies
Runtime/API	Node.js, Express.js, REST APIs, Socket.IO for real-time features.
Data	MongoDB/Mongoose models across services; Redis appears in Auth and cache services.
Messaging	RabbitMQ broker/listener modules for inter-service events.
Security	JWT auth middleware, role-based access control, rate limiting on auth/email routes, validation middleware.
Media/payment	ImageKit/upload middleware for product images; Razorpay payment creation and verification.
AI	Google Gemini REST/LangChain/LangGraph style services, prompt utilities, feature flags, metrics, circuit breaker, retry helpers.
Testing	Jest configs and test suites for auth, product, cart, order, and AI integration scenarios.

8. Status, Setup, and Notes

Current status markers

Area	Status from repository
Core marketplace	In development, with major CRUD and workflow paths implemented.
Payments	Implemented payment service routes including Razorpay and local test success. Some older docs still describe payment as coming soon.
Seller analytics	Seller dashboard routes are present for metrics, conversion, product health, inventory movement, inventory risk, forecasts, and low-stock alerts.
AI	Expanded beyond the original README: chat, product insights, similar products, comparison, budget optimization, mood shopping, feature flags, metrics, and scope endpoints exist.
Documentation drift	Some documentation uses different base paths or older port notes. Route files should be treated as source of truth for implementation details.

Typical local setup

- Install dependencies in each service directory with npm install.
- Configure MongoDB, JWT secrets, RabbitMQ, Redis, ImageKit, Razorpay, email, Google OAuth, and Gemini API keys as required by the services used.
- Start backend services independently through each service's server.js or package scripts.
- Run React frontends through npm run dev in the frontend folders.

- Use the existing Postman/API markdown docs and test suites to exercise service flows.

Recommended demo flow

Sequence	Demo action
1	Register user and seller accounts, then login to capture JWTs.
2	Seller creates products with images and variants.
3	Customer browses products, uses search/filter/wishlist/recent views, and asks AI for product recommendations.
4	Customer adds items to cart, saves one for later, validates cart, and creates an order.
5	Use payment test success or Razorpay verification path.
6	Review notifications, seller dashboard metrics/feed, and low-stock/inventory analytics.
7	Admin adjusts homepage content and notification operations.

9. Source Files Used

This PDF was generated from repository documentation and implementation files, including:

- PROJECT_SUMMARY.md, USER_COMPLETE_GUIDE.md, SELLER_COMPLETE_GUIDE.md, PROJECT_POSTMAN_API_DOCUMENTATION.md
- AI/README.md and AI/src/routes/ai.routes.js
- Auth/src/Routes/auth.routes.js
- product/src/routes/product.routes.js and product/frontend documentation
- cart/src/routes/cart.routes.js
- order/src/routes/order.routes.js and order/API_REFERENCE.md
- Payment/src/routes/routes.js
- Notification/SRC/routes/notification.routes.js
- Seller_dashboard/src/routes/seller.routes.js

The route files were used to resolve differences where older documentation and implementation names did not fully match.